Chapter 3: Data Visualization

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Pandas provides a number of basic plotting capabilities. Matplotlib however gives you more control over details of the visualisation.

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate.

Import required packages

Make sure MLBA package is available

In [12]:
```
pip install mlba
```

```
Requirement already satisfied: mlba in /opt/anaconda3/lib/python3.13/site-pa
ckages (2.0.2)
Note: you may need to restart the kernel to use updated packages.
```

In [13]:
```
%matplotlib inline
import calendar
import os
import matplotlib.pyplot as plt
import mlba
import networkx as nx
import numpy as np
import pandas as pd
import seaborn as sns
from pandas.plotting import parallel_coordinates, scatter_matrix
from sklearn import preprocessing
```

Data Examples

The Boston Housing data contain information on census tracts in Boston for which several measurements are taken (e.g., crime rate, pupil/teacher ratio). It has 14 variables.

A description of each variable is given in Table 3.1, and a sample of the first nine records is shown in Table 3.2. In addition to the original 13 variables, the dataset contains the variable CAT.MEDV, which has been created by categorizing median value (MEDV) into two categories: high and low.

Table 3.2

In [5]:
```
#VB67
#uploading dataframe
housing_df = mlba.load_data('BostonHousing.csv')
# rename CAT. MEDV column for easier data handling
housing_df = housing_df.rename(columns={'CAT. MEDV': 'CAT_MEDV'})
housing_df.head(9)
```

Out[5]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	L
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	
5	0.02985	0.0	2.18	0	0.458	6.430	58.7	6.0622	3	222	18.7	
6	0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	
7	0.14455	12.5	7.87	0	0.524	6.172	96.1	5.9505	5	311	15.2	
8	0.21124	12.5	7.87	0	0.524	5.631	100.0	6.0821	5	311	15.2	2

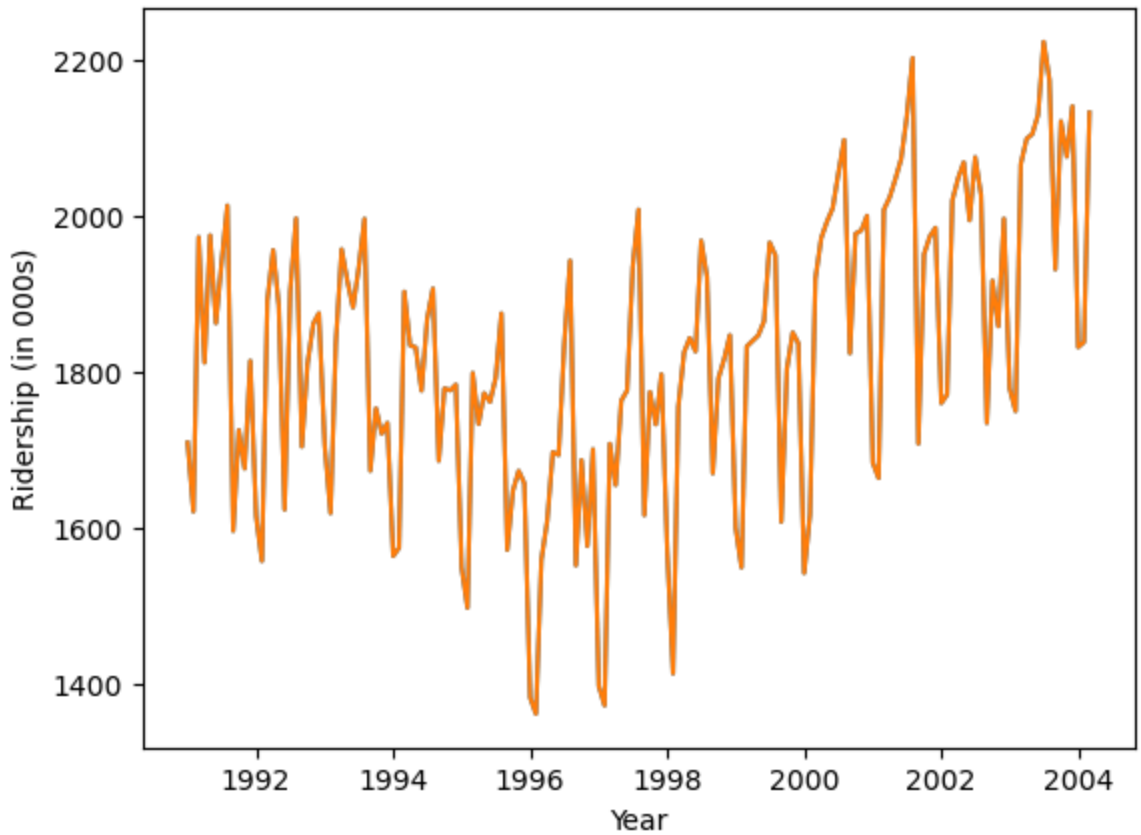
Basic Charts: BAR CHARTS, LINE CHARTS, AND SCATTER PLOTS

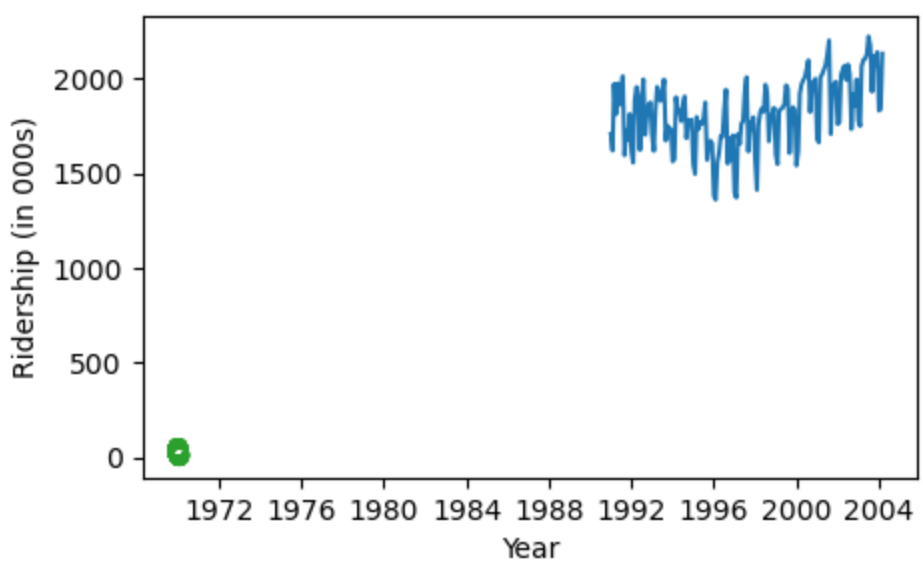
Figure 3.1

Load the Amtrak data and convert them to be suitable for time series analysis. Amtrak, a US railway company, routinely collects data on ridership. Here we focus on forecasting future ridership using the series of monthly ridership between January 1991 and March 2004.

In [9]:
```
#VB67
#uploading the dataset
Amtrak_df = mlba.load_data('Amtrak.csv')
#converting the column Date from the dataframe to specific datetime format
Amtrak_df['Date'] = pd.to_datetime(Amtrak_df.Month, format='%d/%m/%Y')
ridership_ts = pd.Series(Amtrak_df.Ridership.values, index=Amtrak_df.Date)
```

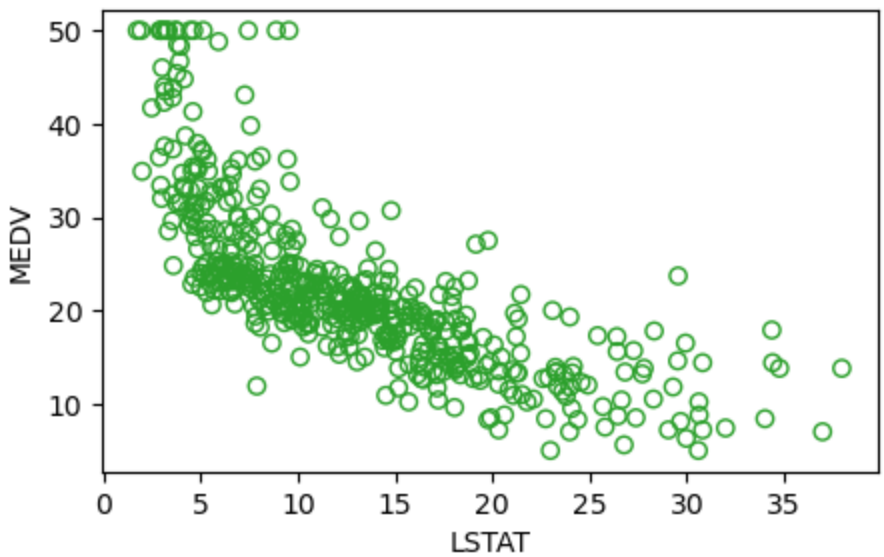
In [14]:
```
#VB67
# plot line graph with matplotlib
plt.plot(ridership_ts.index, ridership_ts)
plt.xlabel('Year')  # set x-axis label
plt.ylabel('Ridership (in 000s)')  # set y-axis label
plt.show()
```



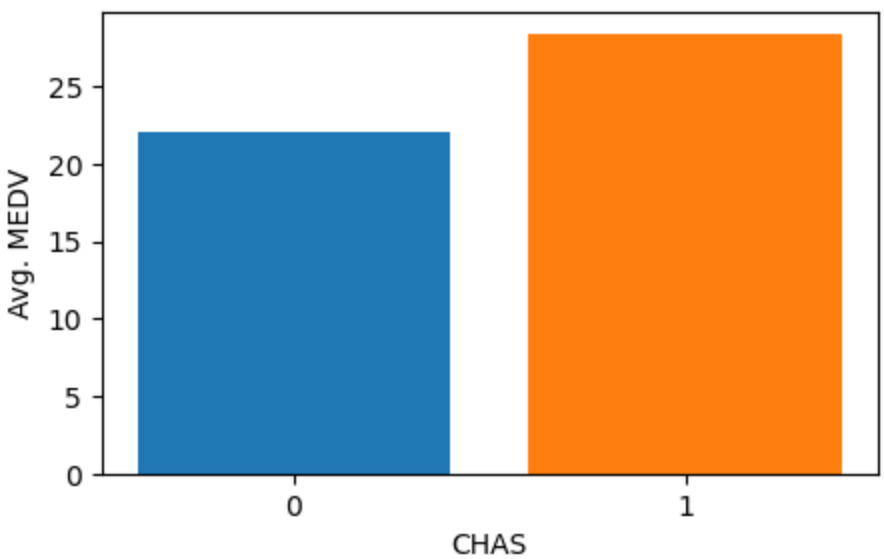


In [15]:
```
#VB67
# generate scatter plot for the Boston Housing Data
# Set the color of the points in the scatterplot and draw as open circles.
fig, ax = plt.subplots(figsize=[5, 3])
ax.scatter(housing_df.LSTAT, housing_df.MEDV, color='C2', facecolor='none')
```

```
plt.xlabel('LSTAT'); plt.ylabel('MEDV')
plt.show()
```

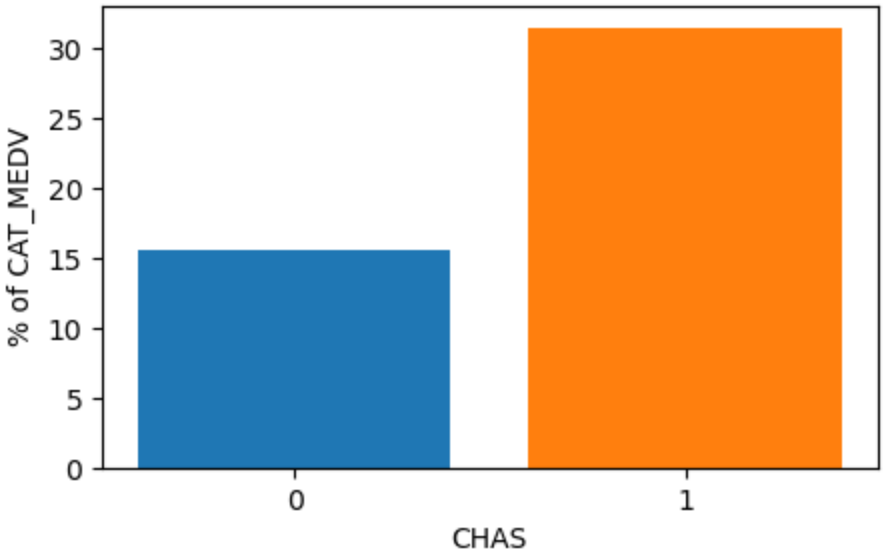


In [16]:
```
#VB67
# barchart of CHAS vs. mean MEDV
# compute mean MEDV per CHAS = (0, 1)
dataForPlot = housing_df.groupby('CHAS').mean().MEDV
fig, ax = plt.subplots(figsize=[5, 3])
ax.bar(dataForPlot.index, dataForPlot, color=['C0', 'C1'])
ax.set_xticks([0, 1], minor=False)
ax.set_xlabel('CHAS')
ax.set_ylabel('Avg. MEDV')
plt.show()
```



In [17]:
```
#VB67
# barchart of CHAS vs. CAT_MEDV
dataForPlot = housing_df.groupby('CHAS').mean()['CAT_MEDV'] * 100
fig, ax = plt.subplots(figsize=[5, 3])
ax.bar(dataForPlot.index, dataForPlot, color=['C0', 'C1'])
ax.set_xticks([0, 1], minor=False)
```

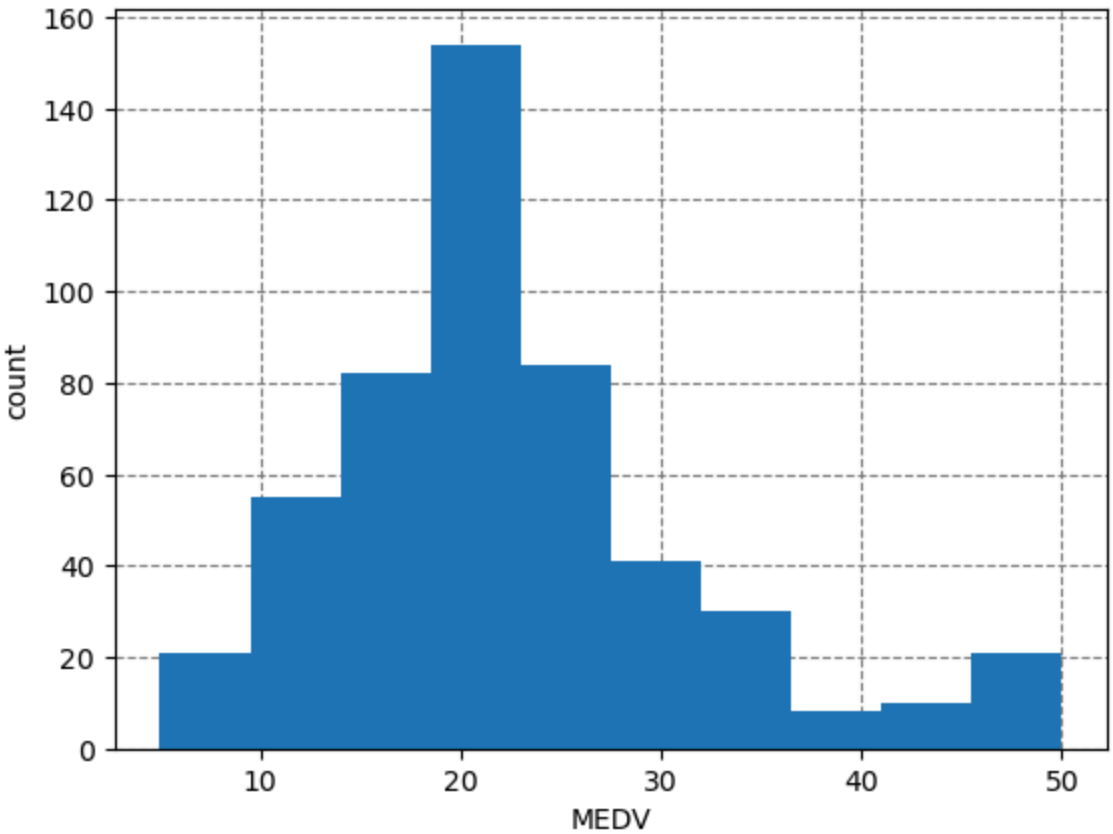
```
ax.set_xlabel('CHAS'); ax.set_ylabel('% of CAT_MEDV')
plt.show()
```



Distribution Plots: Boxplots and Histograms

Figure 3.2

In [18]:
```
#VB67
# Histogram of MEDV
fig, ax = plt.subplots()
ax.hist(housing_df.MEDV)
ax.set_axisbelow(True)  # Show the grid lines behind the histogram
ax.grid(which='major', color='grey', linestyle='--')
ax.set_xlabel('MEDV')
ax.set_ylabel('count')
plt.show()
```



In [37]:
```
#VB67
# boxplot of MEDV for different values of CHAS
ax = housing_df.boxplot(column='MEDV', by='CHAS', figsize=[6, 4.2])
ax.set_ylabel('MEDV')
plt.suptitle('')  # Suppress the titles
plt.title('')
plt.show()
```

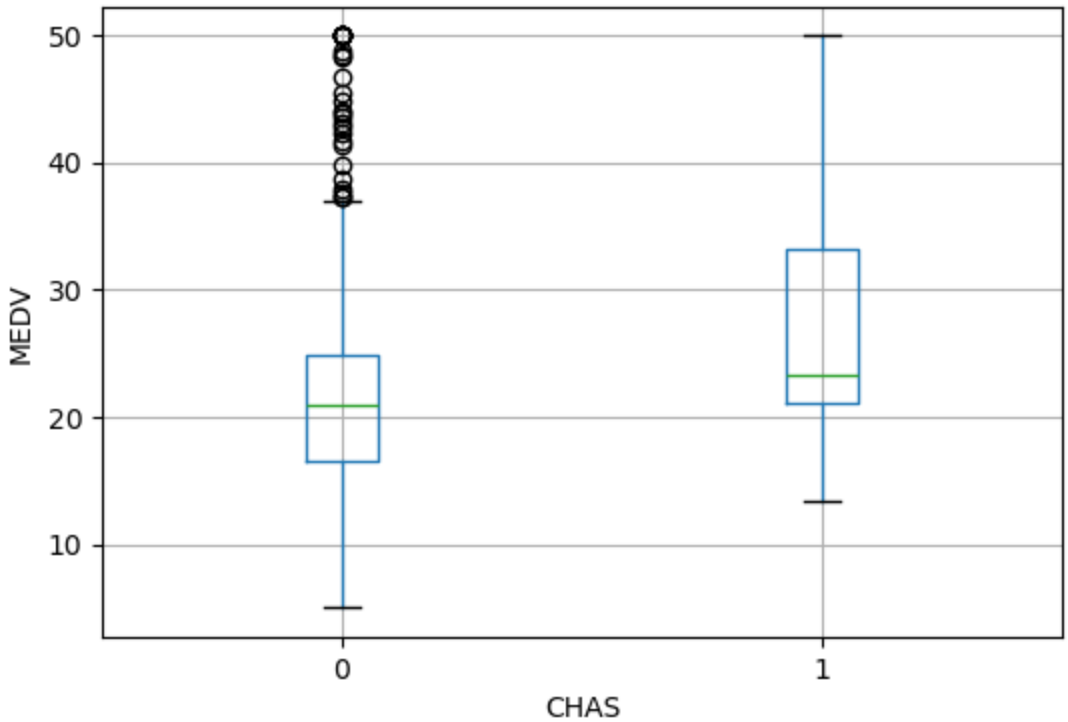
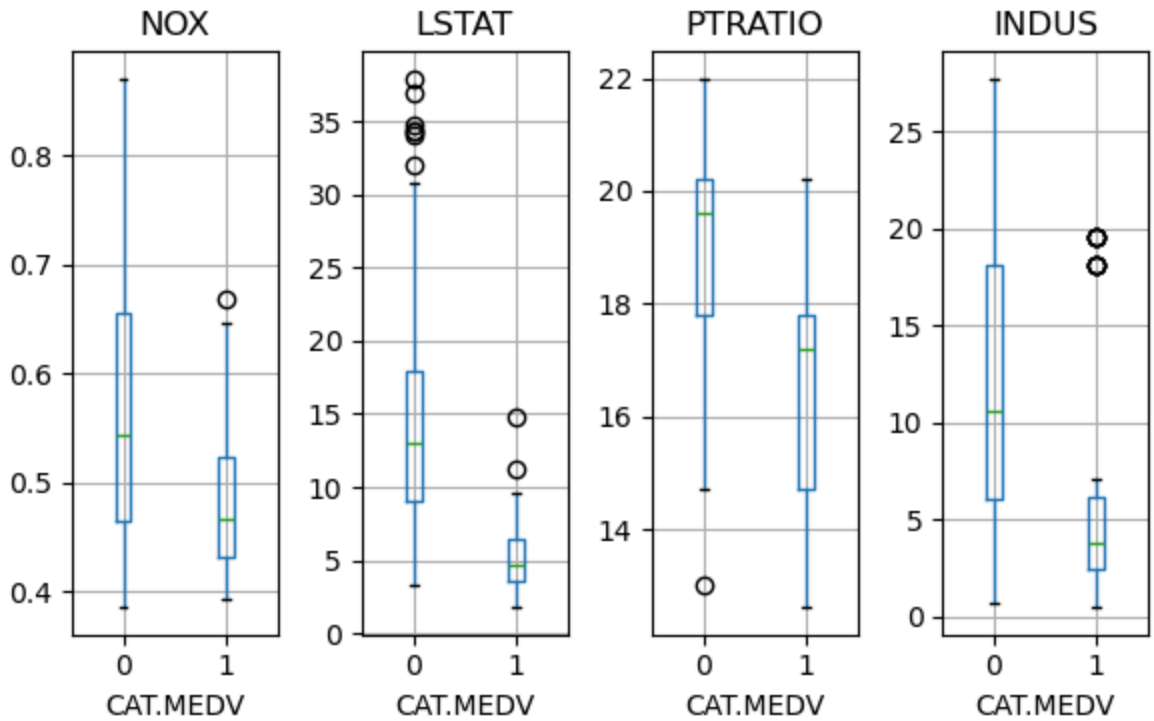


Figure 3.3

Side by side boxplots

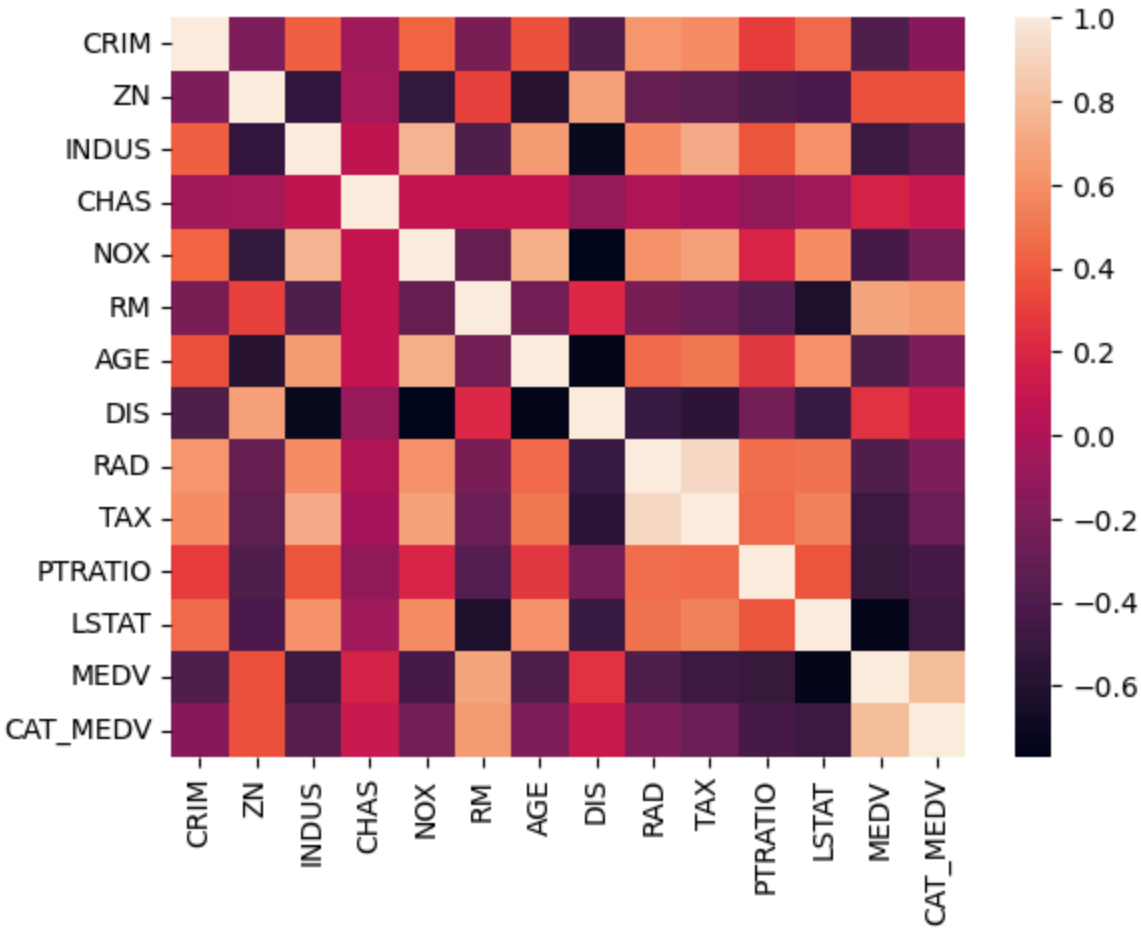
In [20]:
```
#VB67
# side-by-side boxplots
fig, axes = plt.subplots(nrows=1, ncols=4, figsize=[6, 4])
housing_df.boxplot(column='NOX', by='CAT_MEDV', ax=axes[0])
housing_df.boxplot(column='LSTAT', by='CAT_MEDV', ax=axes[1])
housing_df.boxplot(column='PTRATIO', by='CAT_MEDV', ax=axes[2])
housing_df.boxplot(column='INDUS', by='CAT_MEDV', ax=axes[3])
for ax in axes:
    ax.set_xlabel('CAT.MEDV')
plt.suptitle('')  # Suppress the overall title
plt.tight_layout()  # Increase the separation between the plots
plt.show()
```



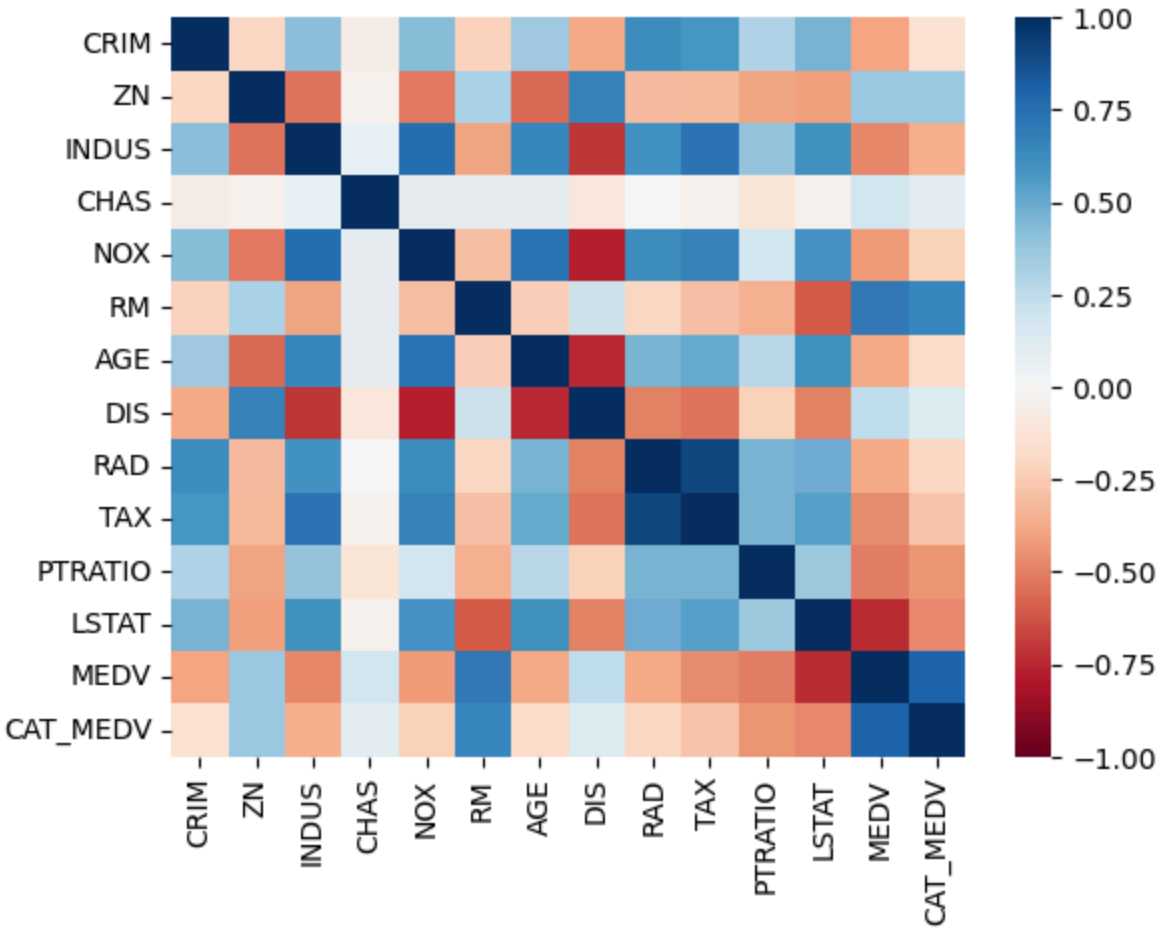
Heatmaps: Visualizing Correlations and Missing Values

Figure 3.4

In [21]:
```
#VB67
# simple heatmap of correlations (without values) with Seaborn (sns)
corr = housing_df.corr()
sns.heatmap(corr, xticklabels=corr.columns, yticklabels=corr.columns)
plt.show()
```



In [22]:
```
#VB67
# Change the colormap to a divergent scale and fix the range of the colormap
# Fixes the color range of the heatmap from -1 to 1 (which is standard for c
# Change color to Blue/Red.
sns.heatmap(corr, xticklabels=corr.columns, yticklabels=corr.columns, vmin=-
plt.show()
```



In [23]:
```
#VB67
# Include information about values (example also demonstrates how to control
# 'annot=True' adds the actual correlation values inside each cell
fig, ax = plt.subplots()
fig.set_size_inches(11, 7)
sns.heatmap(corr, annot=True, fmt=".1f", cmap="RdBu", center=0, ax=ax)
plt.show()
```

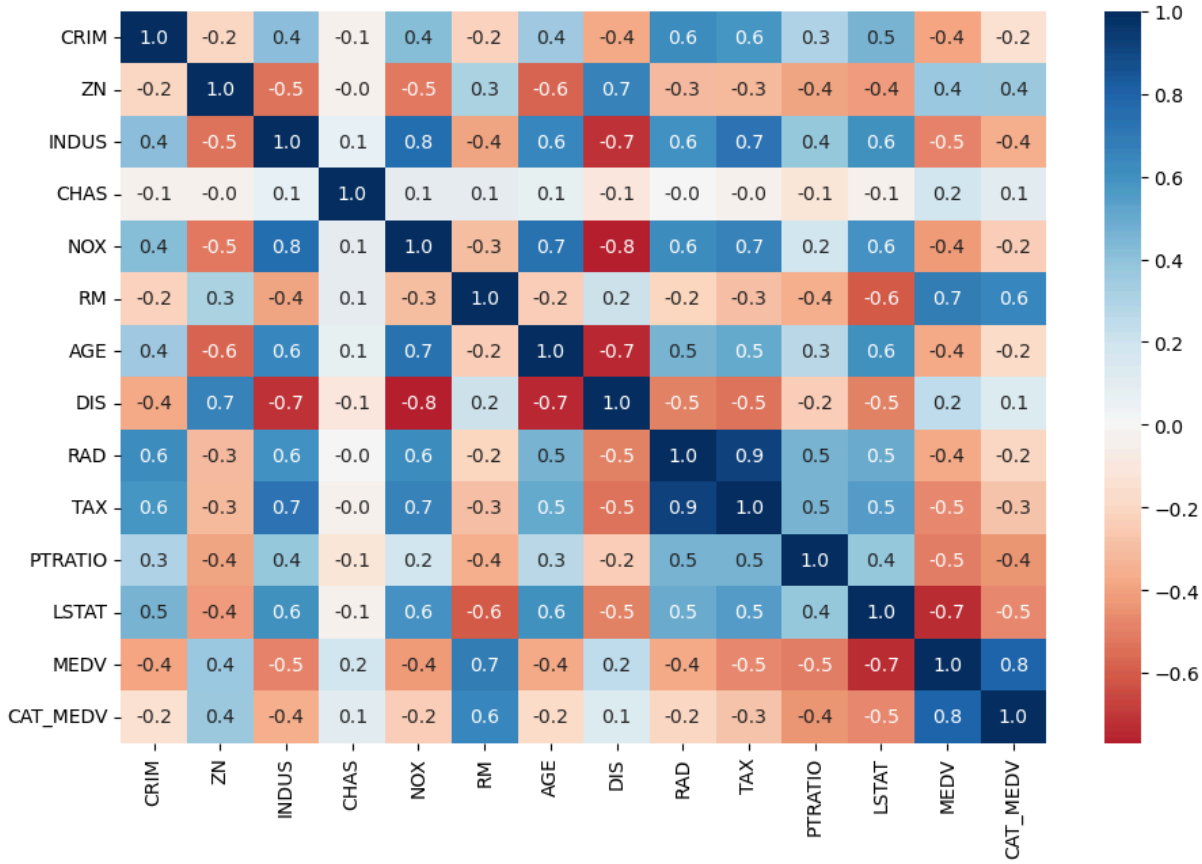
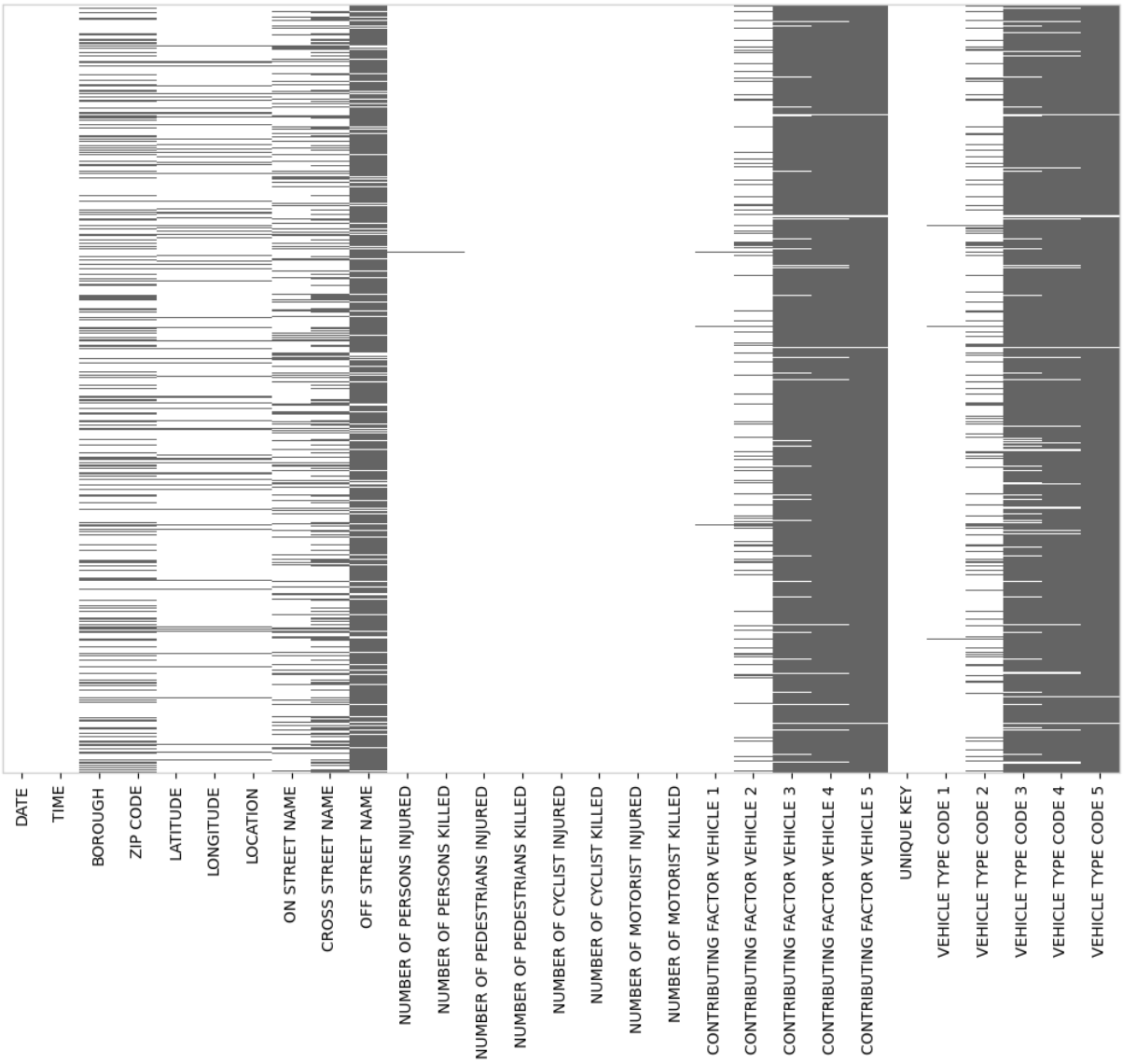


Figure 3.5

A heatmap can also be used to visualize missing values. Here, we use a sample of the NYPD Motor Vehicle Collisions Dataset. (https://data.cityofnewyork.us/Public-Safety/NYPD-Motor-Vehicle-Collisions/h9gi-nx95)

In [24]:
```
#VB67
df = mlba.load_data('NYPD_Motor_Vehicle_Collisions_1000.csv').sort_values(['

# given a dataframe df create a copy of the array that is 0 if a field conta
# value and 1 for NaN
naInfo = np.zeros(df.shape)
naInfo[df.isna().to_numpy()] = 1
naInfo = pd.DataFrame(naInfo, columns=df.columns)

fig, ax = plt.subplots()
fig.set_size_inches(13, 9)
ax = sns.heatmap(naInfo, vmin=0, vmax=1, cmap=["white", "#666666"], cbar=Fal
ax.set_yticks([])

# draw frame around figure
rect = plt.Rectangle((0, 0), naInfo.shape[1], naInfo.shape[0], linewidth=1,
                     edgecolor='lightgrey', facecolor='none')
rect = ax.add_patch(rect)
rect.set_clip_on(False)
```

```
plt.xticks(rotation=90)
plt.show()
```

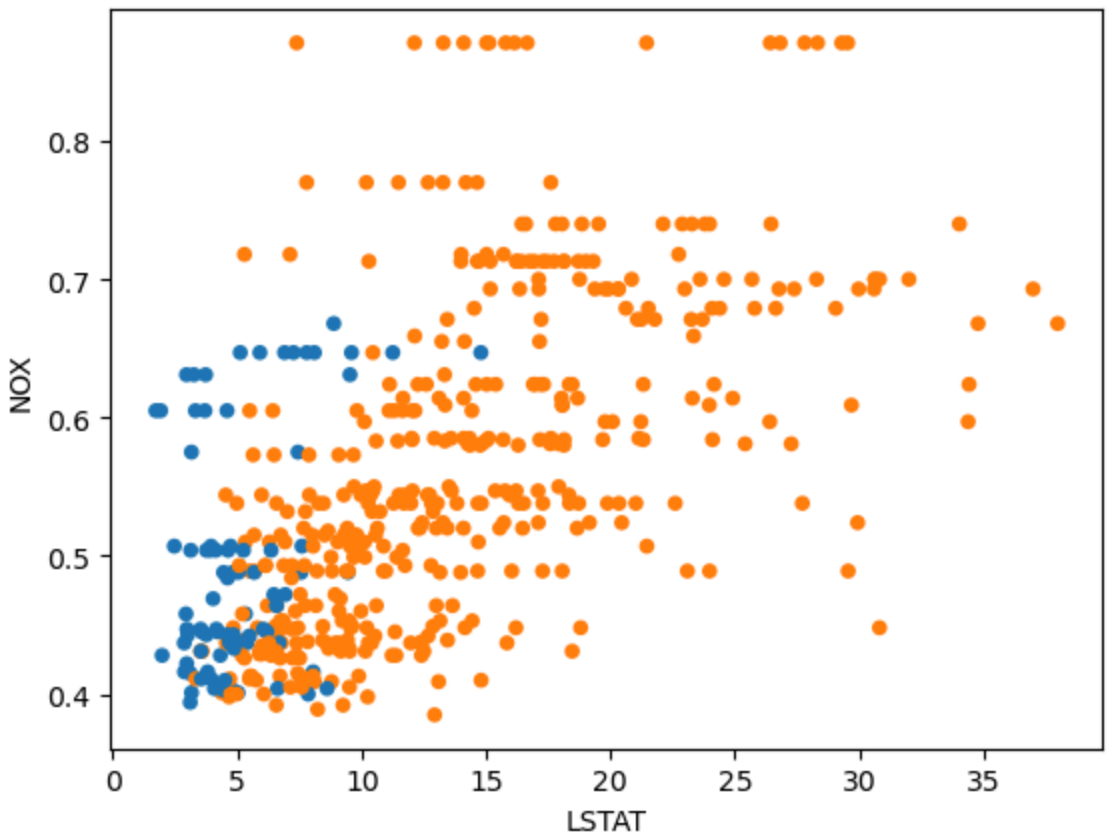


Multidimensional Visualization

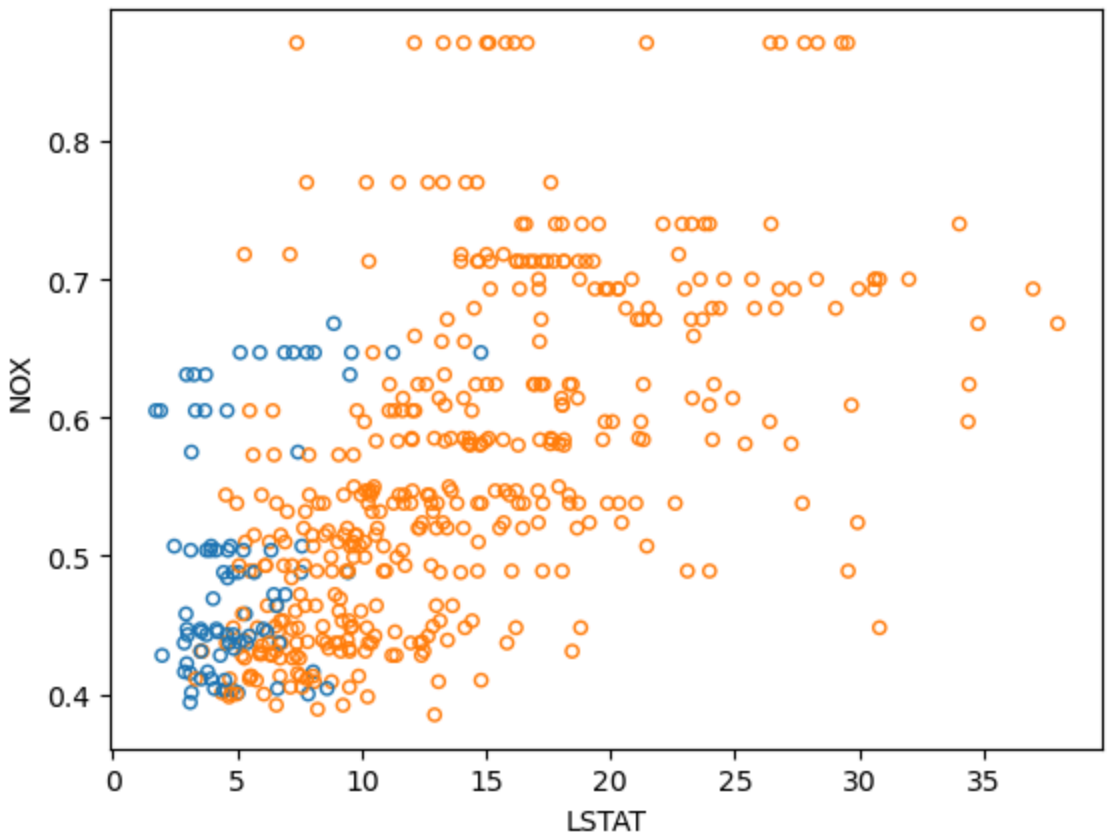
Adding Variables: Color, Size, Shape, Multiple Panels, and Animation

Figure 3.6

In [25]:
```
#VB67
# a scatter plot of two numerical predictors color-coded by the categorical
# Color the points by the value of CAT.MEDV
housing_df.plot.scatter(x='LSTAT', y='NOX',
                        c=['C0' if c == 1 else 'C1' for c in housing_df.CAT_
plt.show()
```

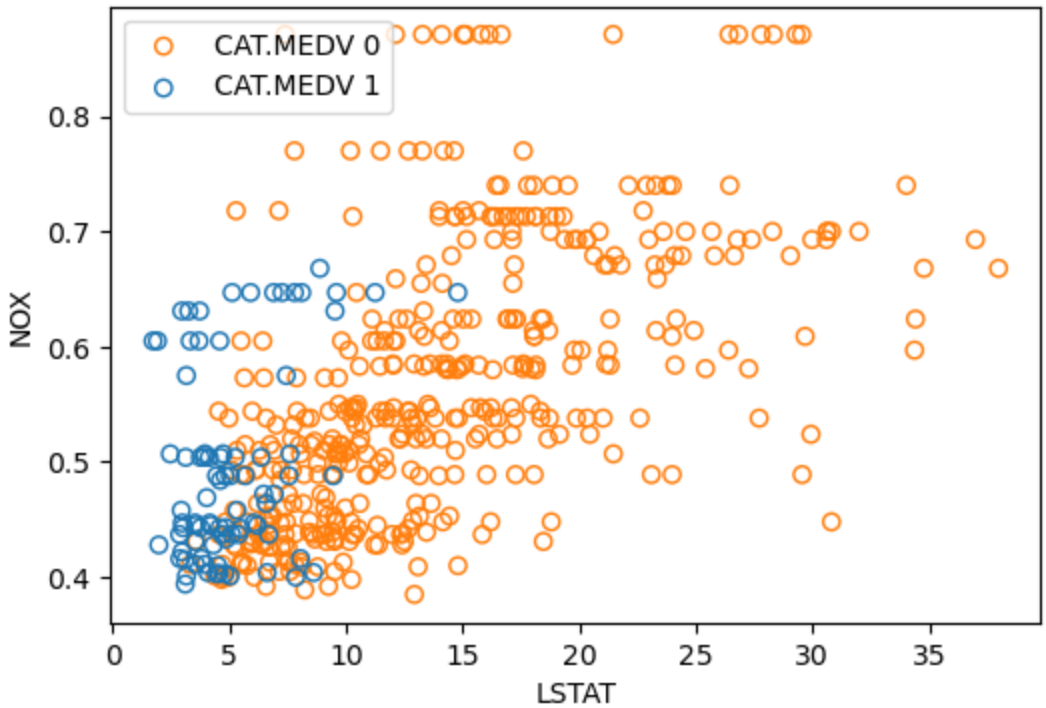


In [26]:
```
#VB67
# Change the rendering of the points to open circles by controlling the colo
housing_df.plot.scatter(x='LSTAT', y='NOX', color='none',
                        edgecolor=['C0' if c == 1 else 'C1' for c in housing
plt.show()
```



If the data points for `CAT_MEDV` of 1 should be emphasized, it's useful to draw them separately.

In [27]:
```
#VB67
# Plot first the data points for CAT_MEDV of 0 and then of 1
# Setting color to 'none' gives open circles
_, ax = plt.subplots(figsize=[6, 4])
for catValue, color in (0, 'C1'), (1, 'C0'):
    subset_df = housing_df[catValue == housing_df.CAT_MEDV]
    ax.scatter(subset_df.LSTAT, subset_df.NOX, color='none', edgecolor=color
ax.set_xlabel('LSTAT')
ax.set_ylabel('NOX')
ax.legend(["CAT.MEDV 0", "CAT.MEDV 1"])
plt.show()
```



Panel plots

Compute mean MEDV per RAD and CHAS and create two bar charts for each value of RAD

In [28]:
```
#VB67
# compute mean MEDV per RAD and CHAS
dataForPlot_df = housing_df.groupby(['CHAS','RAD']).mean()['MEDV']
# We determine all possible RAD values to use as ticks
ticks = set(housing_df.RAD)
for i in range(2):
    for t in ticks.difference(dataForPlot_df[i].index):
        dataForPlot_df.loc[(i, t)] = 0
# reorder to rows, so that the index is sorted
dataForPlot_df = dataForPlot_df[sorted(dataForPlot_df.index)]

# Determine a common range for the y axis
yRange = [0, max(dataForPlot_df) * 1.1]

fig, axes = plt.subplots(nrows=2, ncols=1, figsize=[6, 4])
dataForPlot_df[0].plot.bar(x='RAD', ax=axes[0], ylim=yRange)
dataForPlot_df[1].plot.bar(x='RAD', ax=axes[1], ylim=yRange)
axes[0].annotate('CHAS = 0', xy=(3.5, 45))
axes[0].set_xlabel('')
axes[1].annotate('CHAS = 1', xy=(3.5, 45))
plt.show()
```

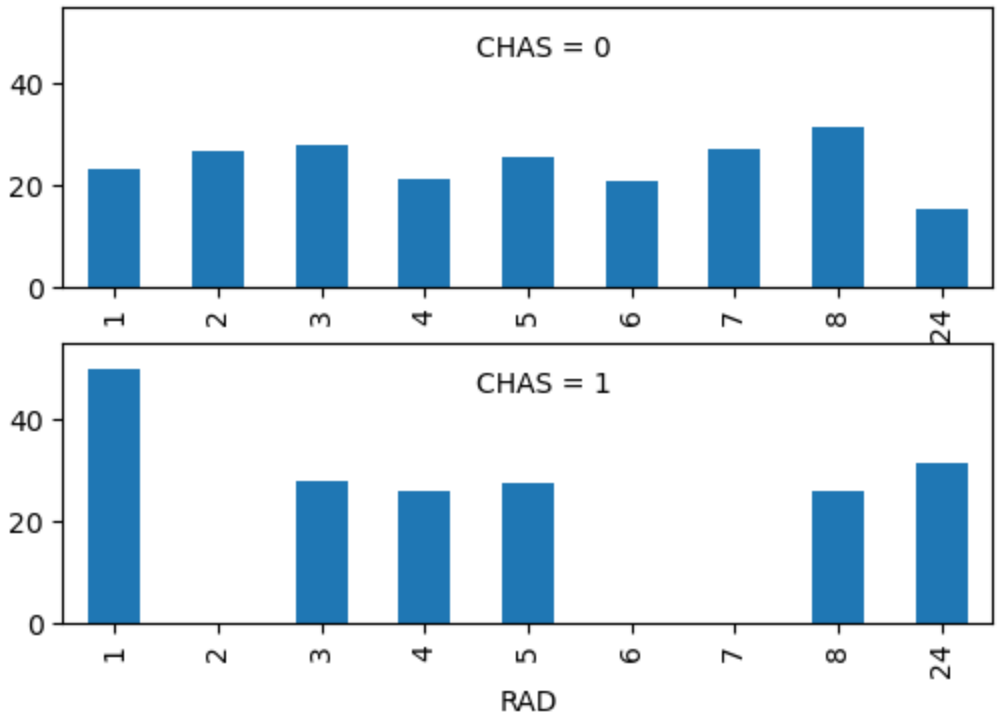
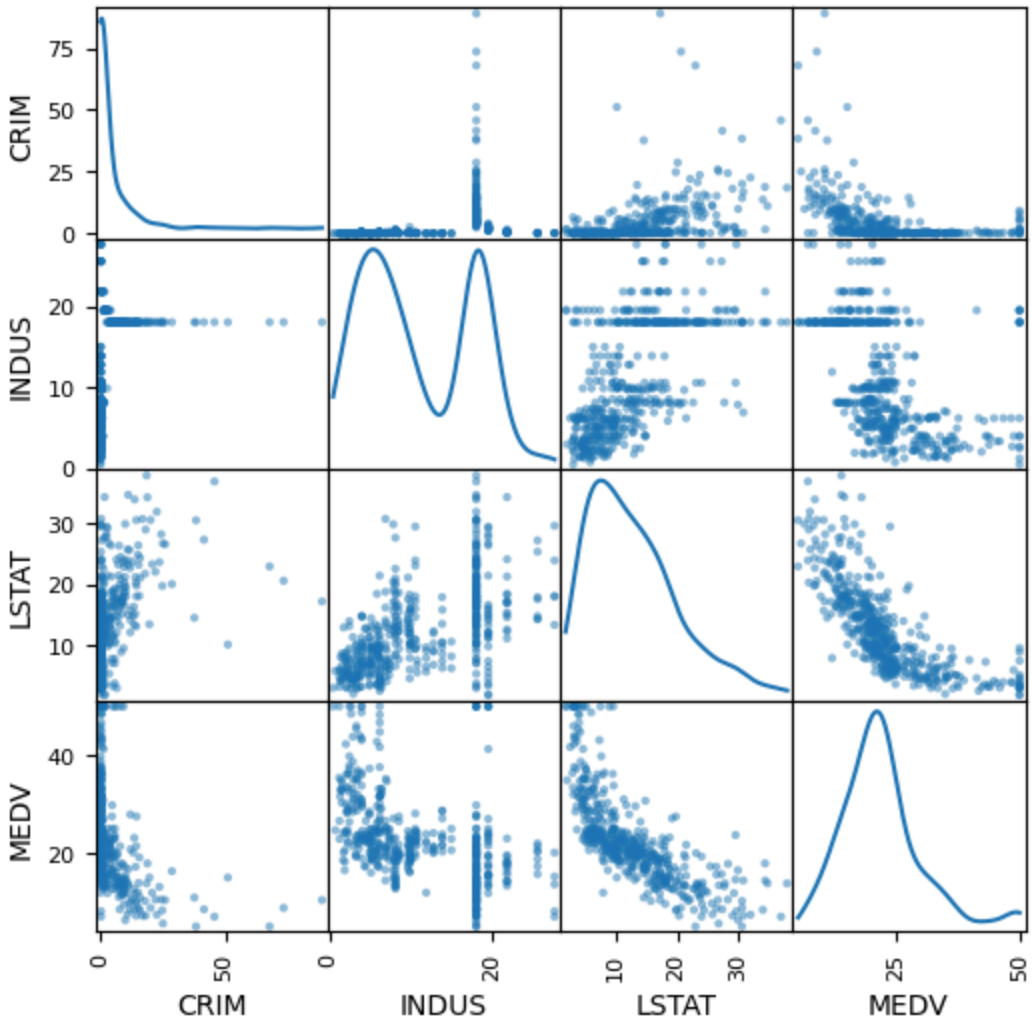


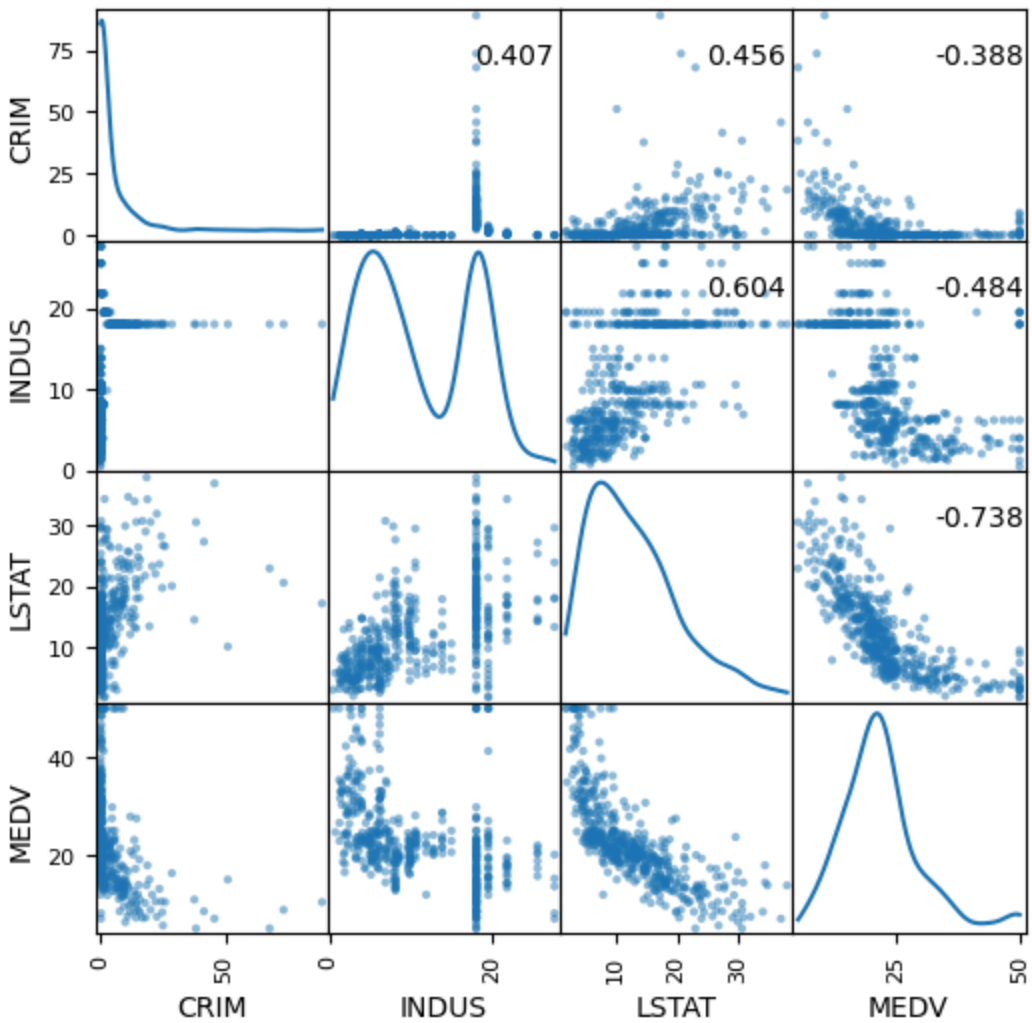
Figure 3.7

Display scatterplots between the different variables. The diagonal shows the distribution for each variable.

In [41]:
```
#VB67
# Display scatterplots between the different variables
_ = scatter_matrix(housing_df[['CRIM', 'INDUS', 'LSTAT', 'MEDV']], figsize=(
```

In [42]:
```
#VB67
# Display scatterplots between the different variables
# The diagonal shows the distribution for each variable
df = housing_df[['CRIM', 'INDUS', 'LSTAT', 'MEDV']]
axes = scatter_matrix(df, alpha=0.5, figsize=(6, 6), diagonal='kde')
# Add the correlation coefficient to the scatterplots above the diagonal
corr = df.corr()
for i, j in zip(*np.triu_indices(len(df.columns), 1), strict=True):
    axes[i, j].annotate(f'{corr.iloc[i,j]:.3f}', (0.8, 0.8),
                        xycoords='axes fraction', ha='center', va='center')
plt.show()
```

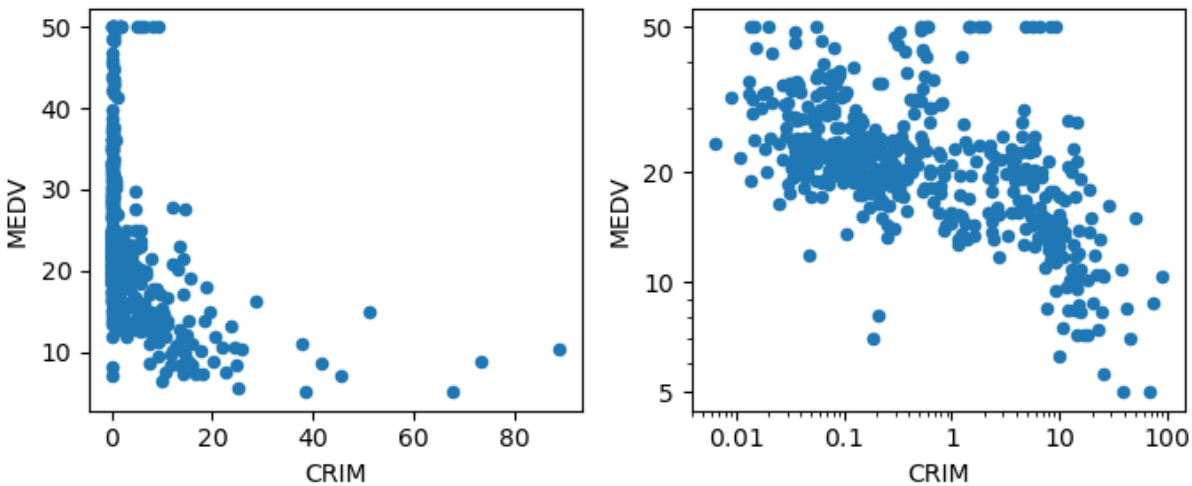

CRIM
INDUS
LSTAT
MEDV
0
25
50
75
10
20
30
40



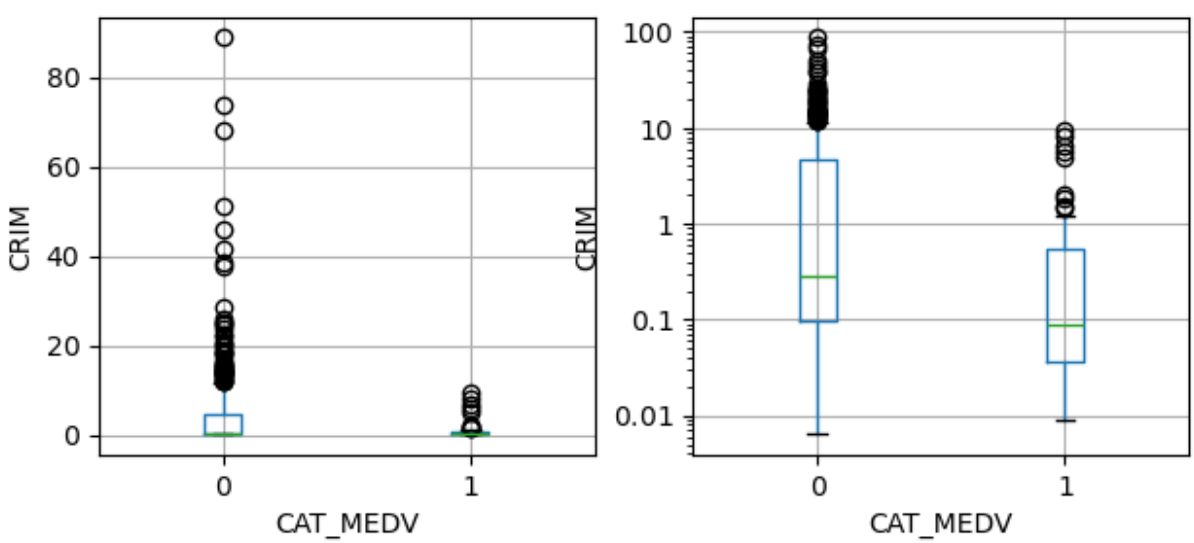
Manipulations: Rescaling, Aggregation and Hierarchies, Zooming, Filtering

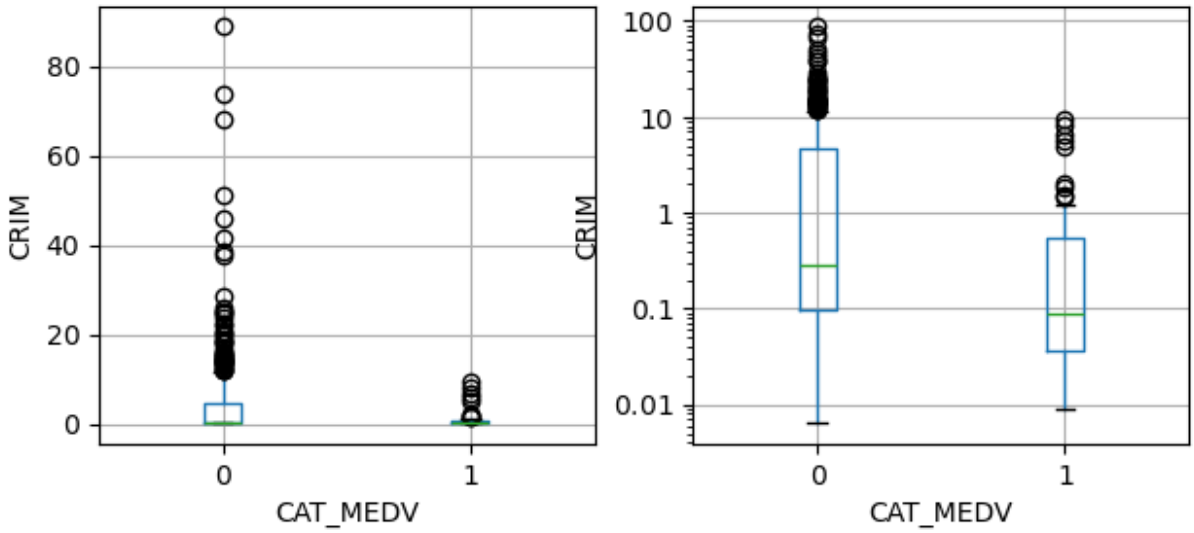
Figure 3.8

In [31]:
```
#VB67
# Avoid the use of scientific notation for the log axis
plt.rcParams['axes.formatter.min_exponent'] = 4

# scatter plot: regular and log scale
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(7, 3))

# Regular scale
housing_df.plot.scatter(x='CRIM', y='MEDV', ax=axes[0])
# log scale
ax = housing_df.plot.scatter(x='CRIM', y='MEDV', logx=True, logy=True, ax=ax
ax.set_yticks([5, 10, 20, 50])
ax.set_yticklabels([5, 10, 20, 50])
plt.tight_layout()

plt.show()
```



In [33]:
```
#VB67
# boxplot: regular and log scale
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(7, 3))

# regular scale
ax = housing_df.boxplot(column='CRIM', by='CAT_MEDV', ax=axes[0])
# log scale
ax = housing_df.boxplot(column='CRIM', by='CAT_MEDV', ax=axes[1])
ax.set_yscale('log')

for ax in axes:
    ax.set_xlabel('CAT_MEDV')
    ax.set_ylabel('CRIM')
    ax.title.set_visible(False)
# suppress the title
axes[0].get_figure().suptitle('')
plt.show()
```





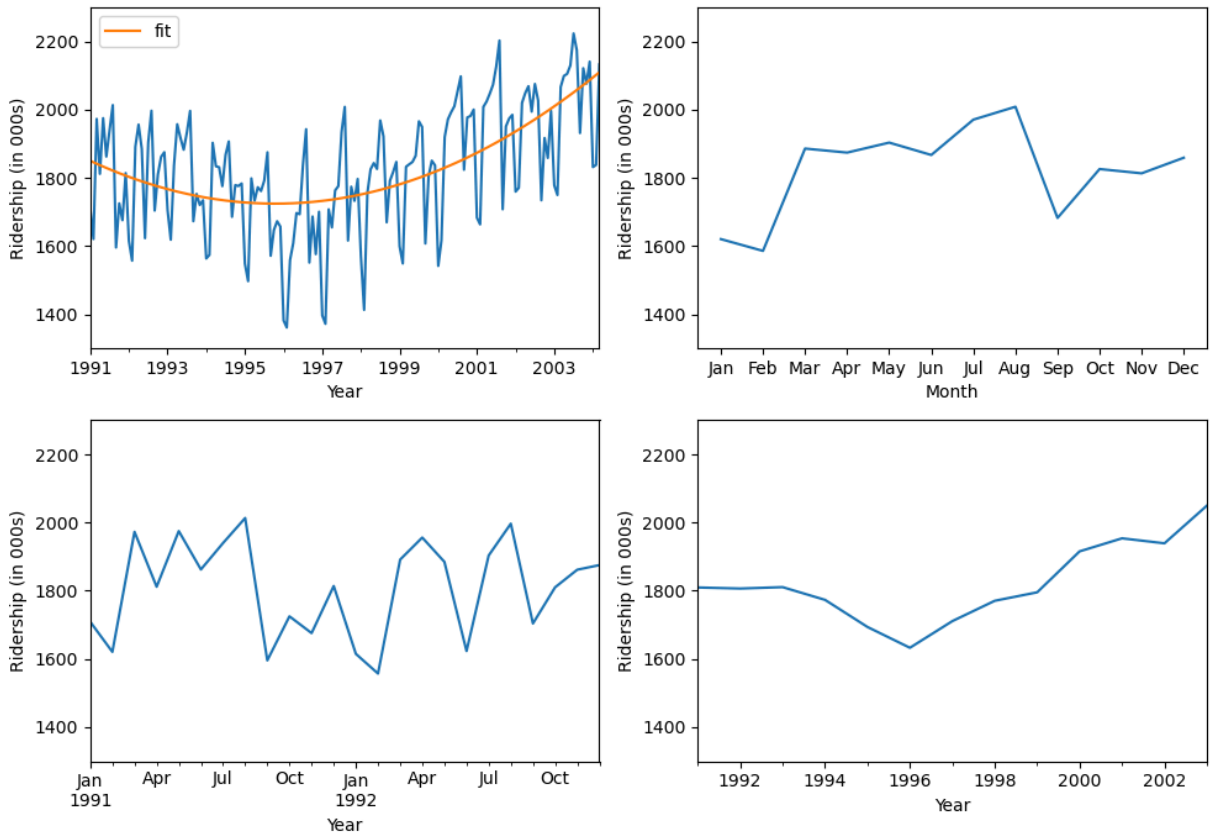
Aggregation and Hierarchies

Figure 3.9

In [34]:
```
#VB67
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 7))

Amtrak_df = mlba.load_data('Amtrak.csv')
Amtrak_df['Month'] = pd.to_datetime(Amtrak_df.Month, format='%d/%m/%Y')
Amtrak_df = Amtrak_df.set_index('Month')

# fit quadratic curve and display
quadraticFit = np.poly1d(np.polyfit(range(len(Amtrak_df)), Amtrak_df.Ridersh
Amtrak_fit = pd.DataFrame({'fit': [quadraticFit(t) for t in range(len(Amtrak
Amtrak_fit.index = Amtrak_df.index

ax = Amtrak_df.plot(ylim=[1300, 2300], legend=False, ax=axes[0][0])
Amtrak_fit.plot(ax=ax)
ax.set_xlabel('Year'); ax.set_ylabel('Ridership (in 000s)')  # set x and y-a

# Zoom in 2-year period 1/1/1991 to 12/1/1992
ridership_2yrs = Amtrak_df.loc['1991-01-01':'1992-12-01']
ax = ridership_2yrs.plot(ylim=[1300, 2300], legend=False, ax=axes[1][0])
ax.set_xlabel('Year'); ax.set_ylabel('Ridership (in 000s)')  # set x and y-a

# Average by month
byMonth = Amtrak_df.groupby(by=[Amtrak_df.index.month]).mean()
ax = byMonth.plot(ylim=[1300, 2300], legend=False, ax=axes[0][1])
ax.set_xlabel('Month'); ax.set_ylabel('Ridership (in 000s)')  # set x and y-
yticks = [-2.0,-1.75,-1.5,-1.25,-1.0,-0.75,-0.5,-0.25,0.0]
ax.set_xticks(range(1, 13))
ax.set_xticklabels([calendar.month_abbr[i] for i in range(1, 13)])

# Average by year (exclude data from 2004)
byYear = Amtrak_df.loc['1991-01-01':'2003-12-01'].groupby(pd.Grouper(freq='A
ax = byYear.plot(ylim=[1300, 2300], legend=False, ax=axes[1][1])
```

```
ax.set_xlabel('Year'); ax.set_ylabel('Ridership (in 000s)')  # set x and y-a

plt.tight_layout()

plt.show()
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_34426/1645486745.
py:31: FutureWarning: 'A' is deprecated and will be removed in a future vers
ion, please use 'YE' instead.
  byYear = Amtrak_df.loc['1991-01-01':'2003-12-01'].groupby(pd.Grouper(freq
='A')).mean()
```



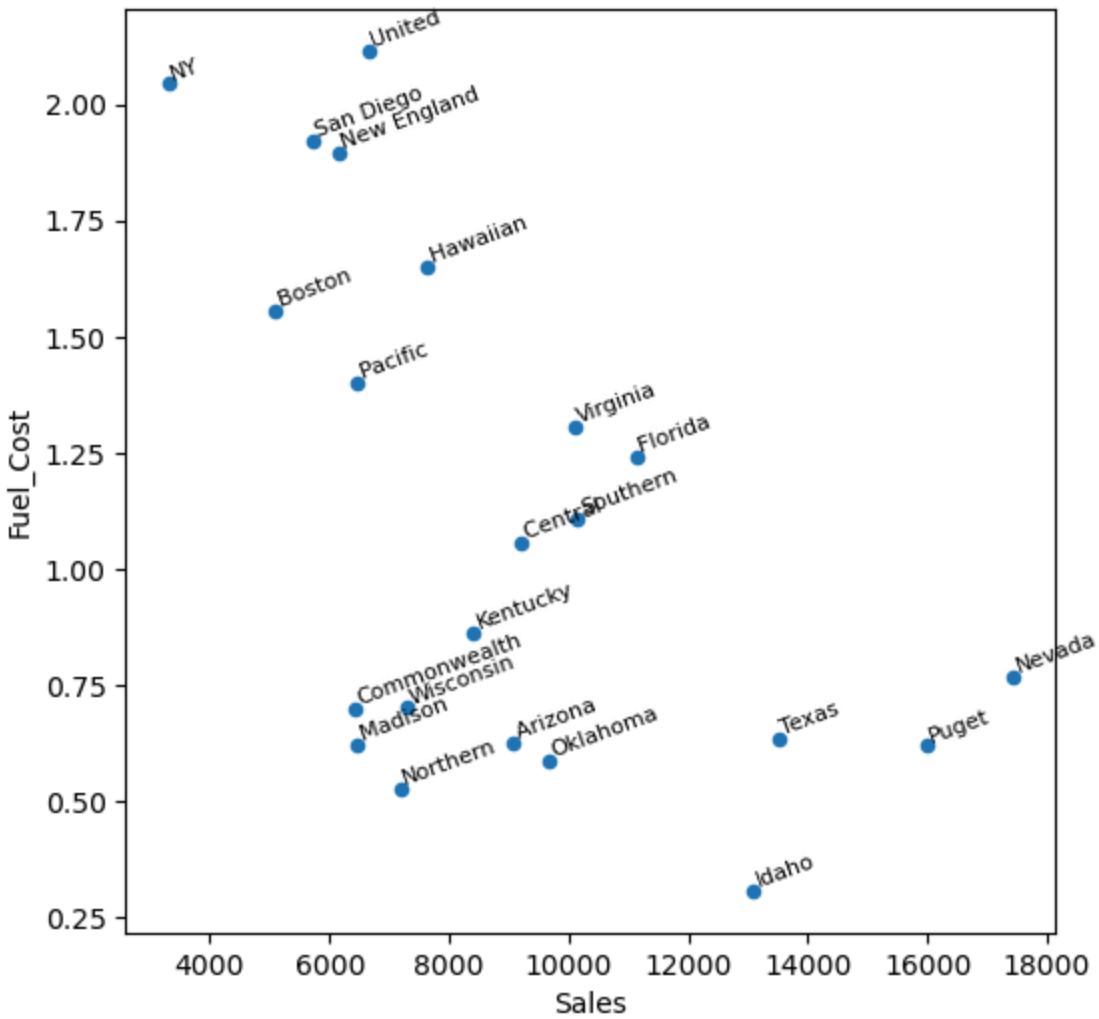
Adding Labels

Figure 3.10

The figure shows different utilities on a scatter plot that compares fuel cost with total sales. We might be interested in clustering the data and using clustering algorithms to identify clusters of utilities that differ markedly with respect to fuel cost and sales. Figure 3.10, with the labels, helps visualize these clusters and their members (e.g., Nevada and Puget are part of a clear cluster with low fuel costs and high sales).

In [43]:
```
#VB67
# loading data to a dataframe
utilities_df = mlba.load_data('Utilities.csv')

ax = utilities_df.plot.scatter(x='Sales', y='Fuel_Cost', figsize=(6, 6))
```

```
# Add labels to the points
points = utilities_df[['Sales','Fuel_Cost','Company']]
_ = points.apply(lambda x:
            ax.text(*x, rotation=20, horizontalalignment='left',
                    verticalalignment='bottom', fontsize=8), axis=1)
plt.show()
```



In [ ]: